

Praktische Anwendung der SysML v2 API am Beispiel von MCAD und Simulation

Marvin Manoury¹, 

Christian Muggeo²

¹ Fraunhofer-Institut für Produktionsanlagen und Konstruktionstechnik IPK, Virtuelle Produktentstehung

² CONTACT Software GmbH, Engineering Transformation

† Diese Veröffentlichung wurde präsentiert und zuerst veröffentlicht beim Tag des Systems Engineering 2023.

Zusammenfassung: Modellbasiertes Systems Engineering als transdisziplinärer Ansatz soll Domänen verbinden – dazu wurde schon mehrfach der Ansatz eines integrierten Systemmodells postuliert. Häufig herrscht jedoch eine Vielzahl an Datensilos der unterschiedlichen Domänen vor, die sich über Importe und Exporte sowie proprietäre Tool-Schnittstellen austauschen. Dadurch entstehen teilweise Datenverluste bei der Konvertierung und hohe Kosten für die Konfiguration und Wartung der Schnittstellen. Die SysML v2 eröffnet mit ihrer API die Möglichkeit, bidirektional mit Systemmodellen zu interagieren. Das bedeutet, dass sowohl Informationen aus dem Systemmodell gelesen, als auch in die Modelle geschrieben werden können. Die Veröffentlichung beleuchtet die API an sich sowie die damit verbundenen Möglichkeiten am Beispiel der MCAD-Konstruktion und der Simulation eines Produktes. Grundlage dafür bildet das Beispiel der Entwicklung eines Staubsaugerroboters.

1. Einleitung

Modellbasiertes Systems Engineering (MBSE) wurde als interdisziplinärer Ansatz zur Entwicklung und Realisierung von Systemen schon in vielen Publikationen präsentiert und Tools, Methoden und Prozesse für den Einsatz im MBSE vorgestellt [1]–[4]. Eine Herausforderung dabei ist immer die Etablierung einer zentralen Informationsquelle und -senke, die durch alle beteiligten Domänen verwendet kann [6]. Häufig wird dafür ein “Systemmodell” herangezogen. Der Begriff Systemmodell beschreibt dabei für technische Systeme mehrere Ebenen [7]: (A) ein domänenspezifischer Teil des Systems oder Subsystems, beispielsweise als Simulationsmodell, (B) eine domänenunabhängige Repräsentation der Struktur und Eigenschaften eines Systems, bspw. als Systemarchitektur oder (C) als Verlinkung verschiedener Systemartefakte. Während Fall (C) als “integriertes Systemmodell” schon seit vielen Jahren versucht wird zu erreichen [8] befinden sich die meisten Umsetzungen auf dem Level der domänenunabhängigen Systemmodellierung (B). Hierfür hat sich die Systems Modeling Language (SysML) als de-facto Standard etabliert.

Die Modellierung der Architektur über SysML sorgt jedoch weiterhin für Herausforderungen: Datenexporte und -importe teilweise als Dokumente oder Tabellen sind ein üblicher Weg der Kollaboration zwischen den Domänen. Datenverluste und die Entwicklung aufwändiger proprietärer Software-Adapter sind hier häufig das Mittel der Wahl [9].

Um Durchgängigkeit zu erreichen sind nach [10] verschiedene Faktoren erforderlich: (a) Daten müssen auffindbar sein, (b) geöffnet werden können wie sie benötigt werden, (c) verarbeitet werden können, sowie ihr Kontext (d) verstanden und (e) vertraut werden können. In [11] wurde eine funktionale Architektur zur Erreichung dieses Ziels vorgestellt, bei der ein zentraler Faktor die Systemmodellierung dar.

Die SysML v2, welche Ende 2022 an die OMG übergeben wurde und sich damit in der Beta-Erprobung befindet, verspricht eine Vielzahl der Herausforderungen adressieren zu können und kann neue Wege einer tatsächlichen Datendurchgängigkeit und interdisziplinären Zusammenarbeit ermöglichen. In dieser Arbeit möchten wir einige Kernpunkte und Möglichkeiten ihres Application Programming Interface (API) beleuchten.

2. SysML v2 und die API & Services

Nach dem Request for Proposal (RfP) im Jahr 2017 wurde eine Vielzahl an Anforderungen zur Verbesserung der SysML v1 gesammelt [12] und ein SysML v2 Submission Team (SST) aufgebaut, um diese Anforderungen umzusetzen, welches bereits 2019 eine “Initial Submission” lieferte. Ergebnis dieser Arbeit sind drei Standards, die sich aktuell in Finalisierung befinden:

- **KerML** [13]: Für die SysML v2 wurde eine eigene Meta-Modellierungssprache geschaffen, die Kernel Modeling Language (KerML). Mit dieser werden syntaktische und semantische Grundlagen für applikationsspezifische Modellierungssprachen geschaffen; Sie stellt eine gemeinsame Grundlage für die Erstellung von domänen-spezifischen Sprachen, wie etwa die SysML dar, die dabei gleichzeitige die Wahrung semantischer Interoperabilität bieten [SysML].
- **SysML v2 Language** [12]: Diese Spezifikation beschreibt die SysML v2 als Modellierungssprache für die Systemmodellierungsdomäne. In diesem Standard werden Syntax und Semantiken für die Systemmodellierung von Anforderungen über Interfaces beschrieben. Darüber hinaus werden erste Domänen-Bibliotheken, wie etwa die “Geometry Domain Library” definiert, mit welcher einfache Geometrien beschrieben werden können.
- **SysML v2 API and Services** [14]: Dieser Standard, als Fokus für diese Veröffentlichung, adressiert die API und Services zur Interaktion mit KerML-Modellen bzw. SysML-Modellen im Spez-

ifischen. Dabei wird ein Plattform-unabhängiges Modell (PIM) sowie zwei Plattform-spezifische Modelle (PSMs) für REST/HTTP und Open Services for Lifecycle Collaboration (OSLC) definiert.

Sowohl für die API und Services als auch die SysML-Modellierungsumgebung existieren Pilot-Implementierungen auf Github: <https://github.com/Systems-Modeling/>.

3. Use Cases

Im Folgenden sollen an zwei Anwendungsfällen Möglichkeiten der SysML v2 API dargestellt werden. Dabei wird der bidirektionale Charakter der API fokussiert, also die Nutzung der Modellelemente als Input für oder Output aus Downstream-Prozessen. Als Modell wird dabei das [RoboticVacuumCleaner Beispiel der GISE MBSE AG](#) in einer generell leicht reduzierten und dafür um Geometrieigenschaften erweiterten Version verwendet. Die relevanten Passagen sind in den folgenden zwei Codeblocks (Code (1)) und (Code (2)) zu finden. Die mit (+) versehenen Zeilen wurden entsprechend ergänzt:

Code (1): VacuumingSystem/BrushSystem

```
17 part def BrushController {
18   port rotationSpeed : BrushRotationSpeed;
19   port brushRotationLevel : ~
    BrushRotationLevel;
20
21   attribute parMaxRPM :> ISQSpaceTime::
    rotationalFrequency;
...
28 }
...
32 part bc : BrushController {
33   attribute redefines parMaxRPM = 167 [min
    ^-1];
34 }
```

Code (2): VacuumingSystem/FilterSystem

```
5 part def Tank {
6   port debris : ~Debris;
7   port fillState : FillState;
(+ 8   item maxBoundingBox : ShapeItems::Box;
9 }
...
24 part tank : Tank {
(+ 25   item redefines maxBoundingBox {
(+ 26     attribute redefines length = 5 [cm];
(+ 27     attribute redefines width = 15 [cm];
(+ 28     attribute redefines height = 5 [cm];
(+ 29   }
(+ 30 }
```

Das Modell wurde mithilfe des %publish Befehls, der in der Pilot-Implementierung des Jupyter-Notebook-SysML-Kernels integriert ist, auf ein öffentliches Repository übertragen. Zum Testen werden drei Anwendungsfälle der “Conformance Test Suite” aus Anhang A der Spezifikation getestet [14]:

- PIM-PS-002 (Get Projects): “Übersicht vorhandener Projekte
- PIM-EN-001 (Get Elements): Adressierung ausgewählter Elemente

- PIM-PCB-010 (Create Commit): Schreiben der Modifikation in Modell-Repository

Für Beispiele der generellen Interaktion mit der API sei hier auf das SysML v2 API Cookbook im Systems-Modeling-Github-Repository verwiesen. Im Folgenden sollen mögliche Interaktionen mit der API an zwei Use Cases dargestellt werden.

3.1. Integration zur Simulation für allgemeine REST-API Interaktion

Als einfaches Beispiel wird die Nutzung der API für eine analytische Berechnung als Ersatz-Simulation in Jupyter Notebook dargestellt. Der Anwendungsfall ist dabei wie folgt:

- Lesen des SysML v2-Projektes aus dem Repository (PIM-PS-003) und Erfassung eines Bauteil-Elements (PIM-EN-001) und seiner Parameter
- Nutzung des Parameters für eine einfache Berechnung
- Schreiben des Berechnungs-Ergebnisses in das SysML-Repository (PIM-PCB-010)

Durch die Abfragen des spezifischen Projektes zum Test von PIM-PS-003 kann auch die entsprechend aktuelle Commit-ID erfasst werden. Für die Erfassung der Parameter der part usage bc wird zunächst die ID von bc mit einer query (Code (3)) bestimmt (PIM-EN-001) und anschließend das Element abgefragt (PIM-EN-002) und seine Parameter bis auf das Level der LiteralInteger identifiziert (Code (4)).

Code (3): Query-body

```
1 {
2   '@type': 'Query',
3   'select': ['name',
4     '@id',
5     '@type',
6     'owner'],
7   'where': {
8     '@type':
9     'CompositeConstraint',
10    'operator': 'and',
11    'constraint': [
12      {
13        '@type':
14        'PrimitiveConstraint',
15        'inverse': False,
16        'operator': '=',
17        'property': 'name',
18        'value': bc
19      }
20    ]
21  }
22 }
```

Code (4): Parameter als Key-Value-Pairs

```
1 def get_subelement(project_id, commit_id,
2   element_id, subelement):
3   element_url = f"{host}/projects/{project_id}
4     }/{commit_id}/elements/{
5     element_id}"
6   return requests.get(element_url).json()[
7     subelement]
8
9 def get_id_type(project_id, commit_id,
10  searched_id):
```

```

6  search_url = f"{host}/projects/{project_id}/commits/" + \ + f"{commit_id}/elements/{searched_id}"
7  return requests.get(search_url).json()['@type']
8
9  def get_value_for_attribute(project_id, commit_id, attribute_id, attr_value_pairs={}):
10 :
11     op_url = f"{host}/projects/{project_id}/commits/" + \ + f"{commit_id}/elements/{attribute_id}"
12     attribute_data = requests.get(op_url).json()
13     attribute_name = attribute_data['name']
14     attribute_member = attribute_data['member']
15     for m in attribute_member:
16         curr_m_type = get_id_type(project_id, commit_id, m['@id'])
17         if curr_m_type == "AttributeUsage" or curr_m_type == "ItemUsage":
18             get_value_for_attribute(project_id, commit_id, m['@id'], attr_value_pairs)
19         elif curr_m_type == "OperatorExpression":
20             arguments = get_subelement(project_id, commit_id, m['@id'], 'argument')
21             for a in arguments:
22                 curr_arg_type = get_id_type(project_id, commit_id, a['@id'])
23                 if curr_arg_type == 'LiteralInteger':
24                     attr_value = get_subelement(project_id, commit_id, a['@id'], 'value')
25                     attr_value_pairs[attribute_name] = attr_value
26     return attr_value_pairs

```

Unter Nutzung des query-Endpunkts wird als ID für bc zurückgegeben, und mit der Eingabe der ID in der Funktion `get_value_for_attribute` wird das Key-Value-Pair `{'parMaxRPM': 167}` zurückgegeben. Dieses wird anschließend für eine einfache Berechnung (Code (5)) verwendet und der Wert in einen Commit-Body gepackt (Code (6)).

Code (5): Berechnung

```

1 n = attribute_val['parMaxRPM'] # Drehzahl aus SysML, min^-1
2 P = 5000 # Leistung
3 M = round(9550*P/1000/n) # Gerundetes Drehmoment

```

Code (6): Commit Body

```

1 { "@type": "Commit", "change": [ {
2     "@type": "DataVersion", "payload": {
3         "@type": "LiteralInteger", "value": M } } ] }

```

Durch die Nutzung des Commit-Endpunktes wird der Parameter als Commit in das Projekt zurück geschrieben.

3.2. Integration zu CAD

Die Integration mit mechanischen Computer-Aided Design (MCAD) wurde bisher vorrangig durch Export/Import oder eigene SysML-Profile und damit verbundene Übersetzungstools umgesetzt [11], [15]–[17]. Die SysML v2 bringt eine Domänenbibliothek für CAD mit, mit der sich bereits einfache Geometrie definieren lassen. Im folgenden Fall wird die API direkt aus dem Programm FreeCAD heraus in einem python-Macro verwendet.

Dabei kann mit den in Use Case 1 beschriebenen Ansätzen einfach das das ey-value-Paar `{'length': 5, 'width': 15, 'height': 5}` identifiziert werden. Diese können in der API von FreeCAD wie in Code-Block Code (7) dargestellt verwendet werden, umdirekt ein 3D-Modell einer Bounding-Box zu erzeugen (siehe Abbildung 1).

Code (7): FreeCAD Code

```

27 def get_value_for_attribute(project_id,
28 commit_id, attribute_id, indent,
29 attribute_value_pairs={}):
30     operator_url = f"{host}/projects/{project_id}/" + \
31         f"commits/{commit_id}/elements/{attribute_id}"
32     response = requests.get(operator_url)
33     ...
34     project_id = "05d56e26-c88c-4f22-baed-92fff88d3b36"
35     commit_id = "985a49f6-d0b1-451c-9dc8-82b9e16e92b7"
36     part_id = "54946a58-1082-43da-8035-79d9db924eb7"
37     attribute_val = {}
38     get_value_for_attribute(project_id, commit_id,
39                             part_id, " ", attribute_val)
40
41     # FreeCAD-specific creation of box
42     doc = FreeCAD.newDocument()
43     valLength = attribute_val['length']
44     valWidth = attribute_val['width']
45     valHeight = attribute_val['height']
46     name = "tank"
47     boundingBox = doc.addObject("Part::Box", name)
48     boundingBox.Height = valHeight
49     boundingBox.Width = valLength
50     boundingBox.Length = valWidth
51     vo = boundingBox.ViewObject
52     vo.Transparency = 80

```

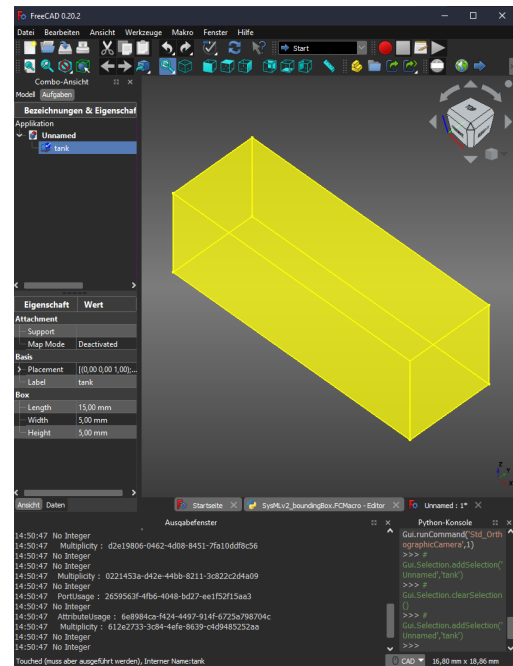


Abbildung 1: FreeCAD-Beispiel

4. Zusammenfassung

In der Publikation wurde die Interaktionsmöglichkeit mit der SysML v2 API am Beispiel einer einfachen Berdechnung und der Integration mit CAD dargestellt. Die Testfälle konnten erfolgreich durchlaufen werden und sowohl upstream als auch downstream Interaktion dargestellt werden. Aufgrund des Umfangs der Publikation konnten nur zwei einfache Beispiele dargestellt werden – die Möglichkeiten gehen jedoch noch wesentlich weiter, etwa durch eine automatisierte Anforderungsprüfung in der Systementwicklung oder der unternehmens- und toolübergreifenden Systementwicklung in einem gemeinsamen Systemmodell. Die Erkenntnisse zeigen, dass die API eine sehr gute Möglichkeit schafft, die SysML v2 für ein tatsächlich interdisziplinäres bzw. transdisziplinäres MBSE stärker in die Praxis zu bringen.

Literaturverzeichnis

- [1] J. A. Estefan, “Survey of Model-Based Systems Engineering (MBSE) Methodologies,” International Council on Systems Engineering (INCOSE); International Council on Systems Engineering (INCOSE)., Seattle, WA, USA, INCOSE-TD-2007-003-02, 2008.
- [2] J. A. Estefan and T. Weikiens, “MBSE methodologies,” in *Handbook of model-based systems engineering*, Springer International Publishing, 2022, pp. 1–40. doi: [10.1007/978-3-030-27486-3_12-1](https://doi.org/10.1007/978-3-030-27486-3_12-1)
- [3] S. Husung, G. Lindemann, S. Korobov, M. Hamester, and S. Kleiner, “Use case driven model-based systems engineering for industrial applications,” in *EMEASEC 2018 / TdSE 2018*, 2018.
- [4] M. Samarajiwa and P. Lünemann, “Die synergie von MBSE und digitalen zwillingen,” in *Tag des systems engineering (TdSE) 2021*, 2021.
- [5] M. M. Schmidt and R. Stark, “Model-based systems engineering (MBSE) as computer-supported approach for cooperative systems development,” in *Proceedings the 18th european conference on computer-supported cooperative work: TheInternational venue on practice-centred computing on the design of CooperationTechnologies - exploratory papers, reports of the european society for SociallyEmbedded technologies*, 2020. doi: [10.18420/ecscw2020_ep04](https://doi.org/10.18420/ecscw2020_ep04)
- [6] M. Bajaj *et al.*, “MBSE++ — foundations for extended model-based systems engineering across system lifecycle,” in *26th annual INCOSE international symposium (IS 2016) edinburgh, scotland, UK*, 2016.
- [7] M. M. Schmidt, T. C. Zimmermann, and R. Stark, “Systematic literature review of system models for technical system development,” *Applied Science*, vol. 11, no. 7, p. 3014, 2021, doi: [10.3390/app11073014](https://doi.org/10.3390/app11073014)
- [8] M. Bajaj, D. Zwemer, and B. Cole, “Architecture to geometry - integrating system models with mechanical design,” *AIAA Space and Astronautics Forum and Exposition, SPACE 2016*. 2016. doi: [10.2514/6.2016-5470](https://doi.org/10.2514/6.2016-5470). Available: <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85088771602&doi=10.2514%2f6.2016-5470&partnerID=40&md5=336203bc9acd8e6e98653a7c51ff92f1>
- [9] M. Pfenning *et al.*, “Killing the PLM monolith—the emergence of cloud-native system lifecycle management (SysLM),” *Fraunhofer-Publica Verlag*, 2021.
- [10] The National Archives, “What is digital continuity.” Available: <https://www.nationalarchives.gov.uk/information-management/manage-information/policy-process/digital-continuity/what-is-digital-continuity/>
- [11] M. Manoury, “Functional architecture for solution independent realization of digital continuity in system development,” *IIPhDW 2023*, 2023.
- [12] “OMG systems modeling language (SysML) version 2.0,” Object Management Group, Feb. 2023.
- [13] “Kernel modeling language (KerML),” Object Management Group, Feb. 2023.
- [14] “Systems modeling application programming interface (API) and services,” Object Management Group, Feb. 2023.
- [15] R. BARBEDIENNE, “Gestion des interactions pour l’évaluation en phase de préconception, des architectures 3D de systèmes sous contraintes multi-physiques, application à la thermique,” Theses, Université Paris Saclay (COMUE), 2017. Available: <https://tel.archives-ouvertes.fr/tel-01565658>
- [16] R. Brahmi, M. Hammadi, N. Aifaoui, and J.-Y. Choley, “CAD-MBSE interoperability for the checking of design requirements,” *18th IEEE International Multi-Conference on Systems, Signals and Devices, SSD 2021*. pp. 1446–1451, 2021. doi: [10.1109/SSD52085.2021.9429472](https://doi.org/10.1109/SSD52085.2021.9429472). Available: <https://www.scopus.com/inward/record.uri?eid=2-s2.0-85107481306&doi=10.1109%2fSSD52085.2021.9429472&partnerID=40&md5=78b3dbc33f0a8a565ad3fee7ba29d842>
- [17] T. Schumacher, D. Kaczmarek, D. Inkermann, and A. Lohrenge, “Fostering model consistency in interdisciplinary engineering by linking SysML and CAD-models,” 2022.